Deep residual network for segmentation and depth estimation

ARTICLE INFO

Keywords:
Residual network, Upsample mechanism, image segmentation, depth estimation

ABSTRACT

Deep residual representations have become an essential foundation for computer vision, which includes image pretraining, image classification, and image segmentation. Residual networks traditionally refer to models that, given an input, add a residual-processed version of the input to the original input. This structure is well-suited to handling information in an input's internal structure and information from domain-specific structured resources. However, in some recent studies in general-purpose research, such models are prone to representation collapse during training. However, it has been indicated that in some recent studies in general-purpose research, such models are unstable during training. One such scenario is that collapse may occur during the training process. Therefore, I present UpResNet: an image classification model that combines an upsampling mechanism and a residual block. By defining different types of classifiers, it can perform image segmentation and depth estimation tasks. The model also employs a novel classification strategy that expands the dimensionality of the data's labels. Unlike conventional labels, during training, the model fits the values corresponding to the new labels, and the actual classification is performed by taking the index of the maximum value among the model's label outputs. Benchmark results show that, on these challenging tasks, the model I propose is competitive with other state-of-the-art methods.

Related Work

Residual blocks learn image representations from large amounts of unlabeled image data and effectively capture the implicit semantics of pixels through permutations and combinations. Representing images using residual blocks is much more suitable for the recent neural-based deep learning models than the traditional image grayscale representation. Based on residual learning models, the recent deep learning methods have been successfully applied to various tasks in the domains of computer vision(CV). With the rapid advance in deep learning, the image models via the residual mechanism have become an integral part of CV models and have attracted significant attention.

In recent years, several residual block-based models and pretrained residual representations have been made publicly available and successfully applied to many tasks in the domain of computer vision. More recently, the kimi team[1] enhanced the model's performance on downstream tasks by modifying the residual blocks—replacing the traditional linear transformation with deep-dimensional softmax attention and combining it with a transformer. Wazir et al.[2] obtained contextual representations by modifying the input dimensions and matrix multiplication, and constructed the attention mechanism in the form of residuals.

Nevertheless, the residual structure does not have to be limited to the combination of a single linear layer. For instance, hyper connections fall under the category of residual connections, which calculate the given input through two of the predefined components, then add the resulting outputs, and use the outputs of the selected two components to be the input for the final residual transformation that also includes the output generated by the third component. If a component within the residual structure is capable of capturing pixel information and leveraging the combination of the residual's internal structure to enhance the image representation at those important locations, then it has the potential to significantly improve the performance of various computer vision applications. Liu et al.[3] recently proposed a new architecture based on residuals, which can effectively utilize pixel information to enrich the image representation. Unlike traditional residual architectures, the new residual architecture utilizes the given input and performs a dot product operation on the output obtained through the defined internal residual structure to represent the final residual output.

Moreover, the different image models corresponding to various tasks mainly rely on a single source of image databases. Some recent studies have indicated that integrating data sources from different tasks into the same model may not necessarily lead to better results. Since the classifiers corresponding to different tasks are not necessarily the same. For the segmentation task, the label corresponding to each image represents the semantic category of each pixel in that image. For the depth estimation task, the label for each image is the color depth corresponding to its grayscale image. The label format may vary depending on the training strategy. Intuitively, training the model separately for

different tasks might be more likely to improve the overall quality of the model after combination, thereby better serving the downstream applications corresponding to those tasks.

In this work, I propose UpResNet: a new model of image segmentation and depth estimation using the combination of the resblock and upsample mechanism on two different image sources: the SIFTflow[4] dataset and the NYU depth[5] dataset. Specifically, I constructed training and test sets from different datasets according to the corresponding rules, respectively, and then generated labeled data for each set based on the label categories associated with each dataset. Subsequently, I used the defined model to train it on the segmentation task image train dataset and the depth estimate task image train dataset. The proposed models are assessed for both validity and utility on the segmentation and depth estimation tasks. The experimental results show that the model I proposed achieved good performance on all benchmark tasks, indicating that the proposed model can indeed improve the quality of image segmentation and depth estimation and better capture semantic information within pixels.

Method

In this section, I will introduce an approach for image segmentation and depth estimation. The training process consists of two steps: 1) constructing corresponding multi-class binary labels based on image data and storing them in the appropriate format. 2) using a model based on the ResNet architecture and introducing an upsampling mechanism after the residual layers. A schematic overview of our method is shown in Fig. 1.

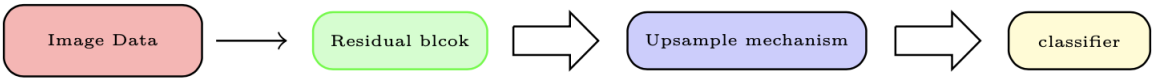


Figure 1: Introduction.

Multi-type binary labels. Inspired by the Binary Cross Entropy(BCE) classifier, I believe that during training, the model does not necessarily have to fit the data directly based on the corresponding label values. Therefore, I expanded the labels of the training data to a 0-1 distribution, where the index of the maximum value corresponds to the true class. This process generated the types corresponding to the labels in the target data. I annotated the type corresponding to each pixel in the image in the type index at the respective pixel location for use in training the model.

I used a standard traversal strategy to generate the labels corresponding to the image—that is, I used a simple ‘for‘ loop to determine the position of each pixel. In the training data, the given image data typically has four dimensions: batch size, channel size, height, and width. For example, color images typically contain three channels, while grayscale images have only one channel. Each image typically corresponds to a label, and the label size usually matches that of the image, with each pixel corresponding to a specific type. In this paper, I correct the dimension of the labels associated with the images—that is, I change the number of channels to match the total number of label types. Then, I mapped the true label to the corresponding channel and set it to 1, while setting all others to 0. Since for each label value, the index of the channel with the maximum value corresponds to its type. Furthermore, the way the model fits the label values during training is actually a variant of Mean Squared Error(MSE), which differs from the conventional cross-entropy loss. This training process should also not be simplistically interpreted to be a form of cross-entropy decoupling, because the formulas corresponding to the models convey different meanings.

In fig. 2, I show an process of the image semantic segmentation. Given an image, the trained model can convert it into an image in the form of a label on the right from the fig 2. Each color in the labels represents a semantic segmentation class. It is obvious from the picture shown that there are two people enjoying a yacht on the sea. The green portions of the labels represent the categories referred to by these two people. Other colors represent other semantic categories, for example, blue represents cruise ships, yellow represents the blue sky, pink represents the ocean, and so on. In the semantic segmentation dataset, I performed a dimension-expansion operation on the labels included in the original dataset to convert them into the format required for training. The model is trained by fitting the values in the labels, thereby avoiding the problems associated with classifying directly based on the raw labels. In addition, there may be many issues with the model making predictions directly using the raw labels. If the labels are too concise, the model may not be able to thoroughly capture the complex data distribution within them, which could make it difficult for the model to converge during training. In this paper, the model primarily consists of a single

component comprising a stack of residual blocks. Stacking different numbers of residual blocks may affect the internal

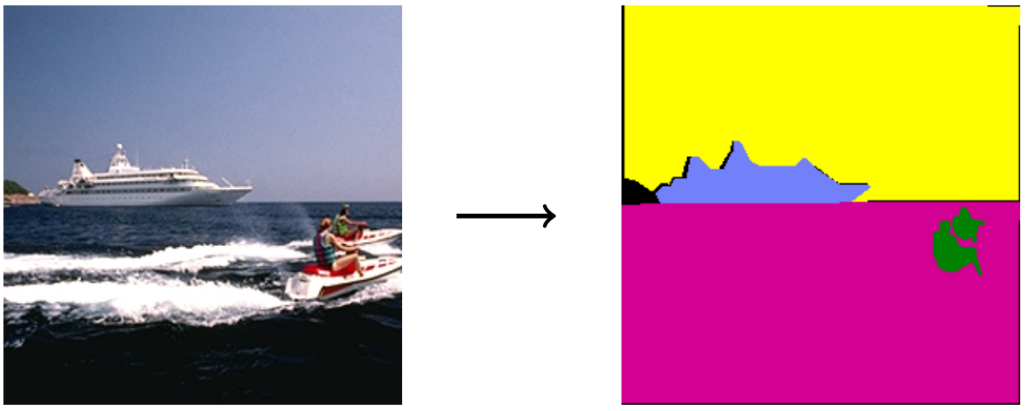


Figure 2: Image Segmentation process.

data distribution of the model across different datasets or models. Therefore, it is important to perform some few-shot learning before training the model. Since it prevents the model's representation from collapsing due to interactions between certain parameters. Another issue that few-shot learning can help prevent is a situation where, for example, a well-defined model initially performs reasonably well but then suffers from model collapse during training due to the influence of certain hyperparameters.

Before the input data enters the model's stacked residual blocks, it must first undergo dimensionality expansion via predefined convolutional operations. The purpose of doing this is, on the one hand, to ensure that the residual blocks in the model align well with the defined labels. On the other hand, dimension-increasing operations allow stacked residual blocks to better capture the semantics of the input data, accelerating model convergence while also improving the model's performance on the test set. Unlike previous studies focused on image segmentation and depth estimation, I perform dimension expansion on the labels associated with the images so that the model can better capture the semantic information within them while learning image representations. Ultimately, when performing segmentation using the dim-expanded labels, the result can be obtained by applying a softmax operation.

Upsample mechanism. Shi et al. proposed an upsampling mechanism via convolution: a ResNet model based on the residual connection that learns the image representation using labeled images, where each low-resolution input image is converted into a high-resolution image. Compared to the ResNet-based model, the model within the upsampling mechanism can make effective use of the image information and its corresponding task label to improve the model quality. In the domain of image segmentation, many entities with the same semantic meaning—which are not actually connected during training—end up being connected, making it difficult for models that rely solely on residual connections to learn them effectively. In contrast, the model is clearly better suited to handling such situations by incorporating an upsampling mechanism. For instance, since people, landscapes, and objects are common in semantic segmentation training datasets, models with upsampling mechanisms can learn representations of these entities based on the images and their labels, and then reproduce them in images of the same size via the input, thereby generating color representations of different entities. In this study, we apply the model with an upsampling mechanism to learn image representation from the image segmentation task and the depth estimation task.

The upsampling mechanism in the model is essentially a combination of subpixel convolution and convolution. Subpixel convolution includes an upsampling factor. After the model sends the given input to the upsampling mechanism, the upsampling mechanism adjusts the input's dimensionality based on the upsampling factor. The change in the sizes of his last two dimensions is typically the current dimension size multiplied by the upsampling factor in the subpixel convolution. This means that the original ResNet-based model can better learn from image data after introducing an upsampling mechanism, enabling it to handle various downstream tasks, for example, image segmentation and depth estimation.

To enable the model to better fit the given images and their label data during the training process. The model's hyperparameters are tuned before it undergoes full training. In some models, there may be situations where the loss value is very close to the convergence threshold but never quite reaches the point of convergence, and where there is significant fluctuation during the training process. Once it has been confirmed that there are no major issues with the interactions between the model's internal components, this situation can be improved by reducing the learning rate, thereby allowing the model to reach convergence. Based on subpixel convolution and standard convolution, given an input, the dimensionality may appear unchanged, but in reality, the internal parameters within the mechanism have undergone substantial changes. The advantage of a residual-based model with an upsampling mechanism is that it can learn image representations from different training datasets more thoroughly, which is highly beneficial for image segmentation and depth estimation tasks.

In this study, the input to the defined model consists of image data from semantic segmentation datasets and depth estimation datasets. After the given image has been processed through the upsampling mechanism, it must be further optimized using convolution and activation functions. Since the model uses a BCE classifier, the magnitude of the model's final output is constrained to the range of 0 to 1 by the activation function. Furthermore, the shape of the dimension must match that of the corresponding label.

The upsampling design aims to ensure that subpixel convolution, when combined with convolution, yields more desirable results. Simply expressing the subpixel convolution through a formula may not necessarily achieve the desired results, because subpixel convolution may not be compatible with other components within the model. After combining it with convolutions and adjusting other parameters in few-shot learning, the model can achieve good results during overall training. In this paper, I apply the model to the image segmentation task and the depth estimation task, respectively.

Implementation details. In my experiment, I downloaded image data files from the Internet based on different computer vision tasks. The siftflow dataset contains 2,688 images, each carefully segmented into 33 semantic categories, and each image is also classified by geo labels into 4 categories. I converted images and their corresponding labels from different computer vision tasks into training-ready labels to build the corresponding datasets. All image data are normlized using CV tools before being fed into the model for training.

For the depth estimation dataset, I downloaded its data from the internet.This depth estimation dataset contains 1,449 RGB images captured from various commercial and residential buildings in different cities, covering 464 distinct indoor scenes across 26 scene categories. A standard grayscale image contains 256 grayscale levels, so I define the images in the depth estimation dataset into 256 categories. Note that depth estimation tasks are an integral part of the field of computer vision and play a crucial role in computer vision applications. For instance, when the model segments a current object, that object may not necessarily be parallel to the camera, so the distance from each point on the object to the camera may not be the same. Therefore, a trained depth estimation model can be used to estimate the distance between the object and the camera, thereby assigning a color depth to improve the object's recognizability. Since the grayscale image corresponding to each image is already provided in the depth estimation task, I only perform dimension expansion on the labels within the given grayscale images. And all image data are also normlized using CV tools before being fed into the model for training.

The model I proposed was trained using an empirical approach with the following hyperparameters. For the model's optimizer, I used the Adam optimizer built into the PyTorch framework. Since setting the learning rate hyperparameter too high can cause the proposed model to collapse during training, I set the learning rate to 0.00001 to prevent this from happening. For tasks across different computer vision domains, I set the number of training iterations for the model to 200 across the board. And the batch size for model training is set to 1 to make it easier to observe the training line chart.

Data Record

The proposed models are commonly used and evaluated in two types of computer vision tasks: segmentation and depth estimation. For image segmentation tasks, the proposed model is used to be a classifier for image segmentation in the field of computer vision. For depth estimation tasks, the proposed model is used to be a classifier for depth estimation in the field of computer vision; this classifier requires prior customization based on the classification type. The proposed model contains stacked residual blocks, and varying numbers of these blocks yield different results during training. In my preliminary experiments, I also observed similar results: when the number of residual blocks was set to 16 and 64, respectively, the proposed models achieved different results. The model with 16 residual blocks

diverged during training, while the one with 64 residual blocks converged. Therefore, in this study, I followed their approach by setting the number of stacked residual blocks to 64 and training the model separately for various tasks in the field of computer vision. During training, the model changes the size of the second dimension of the image data, while the sizes of the other dimensions remain unchanged.

The model I propose can effectively integrate the input image data, thereby improving the representation of such models across various computer vision tasks. In Table 1, I present the results of the proposed model for segmentaion task and depth estimation task. I calculated the number of pixels required for the output of all given images in the evaluation dataset. It can be seen that the models in Table 1 achieved relatively high scores on the dataset in the geo label format with the same number of training iterations. In this study, the model did not incorporate a pre-trained model instead, it was trained directly using data from the pre-segmented training set. Table 1 primarily illustrates whether, for tasks in different domains, given a set of images or a single image, the model correctly identifies the type of each pixel in that image. However, behind these impressive statistics lies a tendency to deceive the public. Some models only demonstrate strong performance at the data level, but may not perform well when applied to specific tasks. For example, although the model's pixel-level accuracy has improved, in terms of segmentation, there may be cases where two entities with the same semantic meaning are merged together.

models	dataset	Acc.
UpResNet	Sift semantic	$\frac{9007222}{13107200}$
UpResNet	Sift geo	$\frac{10519297}{13107200}$
UpResNet	NYU depth	$\frac{66687913}{200908800}$

Table 1: results.

In particular, by displaying the model's training line chart, I observe changes in model performance, which allows me to better understand how the model represents information across different tasks—a finding of immense value for applications in the field of computer vision. Figure 3 uses a line chart to illustrate the relationship between the number of training iterations and the loss function. It is not hard to see that the models exhibit the same downward trend during the training process. The reason I've placed these three curves in separate images is that, the models have different labels, it might be difficult to discern the differences between them if they were all shown in a single image. Since within the same trend, models with higher volatility tend to obscure the changes in models with lower volatility.

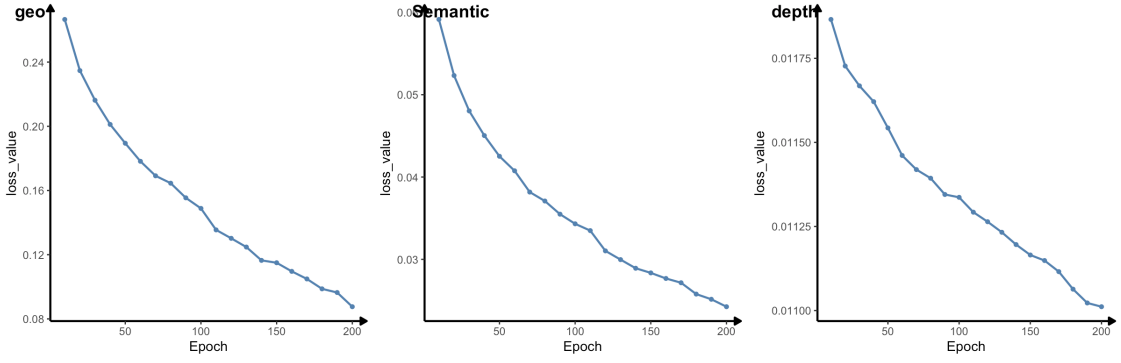


Figure 3: Line chart.

Principal Component Analysis

To better understand the internal parameter structure of the models, I performed principal component analysis on the logit outputs for each model and its corresponding task. Each dataset contains its own dimension-expanded labels, and I calculated the statistical summary of the model's logical predictions for each dataset. During the dimension-reduction process, I first used singular value decomposition to transform the model's output into two unitary matrices and a vector of singular values. Then, I took the first two values from the singular value vector to be the two principal

components. Based on these two principal components, I extracted the values corresponding to these two dimensions from the unitary matrices to be the model's representation of the two principal components for each data point. The proportion of each principal component is the share that component accounts for in the total. Finally, the results are presented in the form of a scatter plot.

In my experiments, since the label formats differ across tasks, I used the same model with different classifiers for each task. From the figure 4, it can be seen that the model made both correct and incorrect predictions on the test set. Research findings indicate that residual networks and upsampling mechanisms are of significant value and highly useful in the field of computer vision. The red dots in the figure represent points where the model made a prediction error, while the light green dots represent points where the model made a correct prediction. The x-axis represents the distribution of the first principal component, and the y-axis represents the distribution of the second principal component.

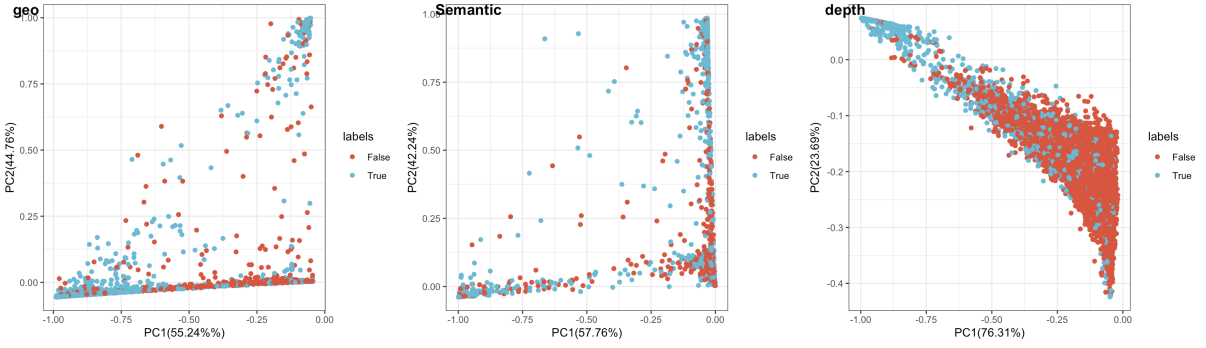


Figure 4: Princinple component analysis.

From figure 4, I can also see that, after dimensionality reduction, the model's results on the test set form a triangular distribution in the image. The reason the model's triangular shape is not clearly defined in the depth estimation dataset is that the number of training iterations did not meet the ideal standard or the training set was too large. After increasing the number of training iterations, the model's output, once reduced in dimension, exhibits a clearer triangular distribution. In addition, I noticed that the points where the model made prediction errors with the corresponding dataset are most clustered in the lower-right corner of the triangle. It's not certain whether it's slightly to the left or slightly to the right of the bottom-right corner.

Logits distribution analysis

This paper demonstrates the application of UpResNet in two different scenarios: image segmentation and depth estimation. Residual networks are commonly used to compute the representation of images within a model. In the general domain, the Image Segmentation and Depth Estimation Challenge has been held for many years. The competition aims to solicit models capable of effectively segmenting images and estimating their depth. Among the various challenges, pixel accuracy is used to be a benchmark to measure the quality of models for image segmentation and depth estimation: each image is normalized before being fed into the model, which then converts the image into logits corresponding to the task at hand; pixel accuracy is actually measured by calculating the difference between the logits and the ground-truth labels, typically using an equal-or-not strategy. In the domain of image segmentation and depth estimation in computer vision, pixel-level accuracy is also crucial. I conducted a case study to investigate the effectiveness of the proposed model in image segmentation and depth estimation tasks. I used image segmentation and depth estimation datasets. The image segmentation dataset includes two types of labels: the "geo" type, which has four categories, and the "semantic segmentation" type, which has 34 semantic classes. The depth estimation dataset's labels include 256 depth levels. For the test sets of different datasets, I recorded the logical values corresponding to the maximum index of the true labels for each trained model. Then I presented it in the form of a violin plot from Figure 5. The violin plot is a type of data visualization chart that combines the characteristics of box-and-whisker plots and kernel density plots to illustrate the distribution of data. Based on the combination of the density plot and its coordinates, the model's logit values for the true indices are either very close to 1 or very close to 0. This phenomenon is not necessarily caused by severe polarization,inadequate model training may also be a contributing factor. Based on a comprehensive analysis of the box plots and the positions of their quartiles, the model performs best on data with

the “geo” label type. Based on the length of the boxes in the box-and-whisker plots, it is not difficult to see that the distribution of the logit values output by well-trained models is relatively concentrated.

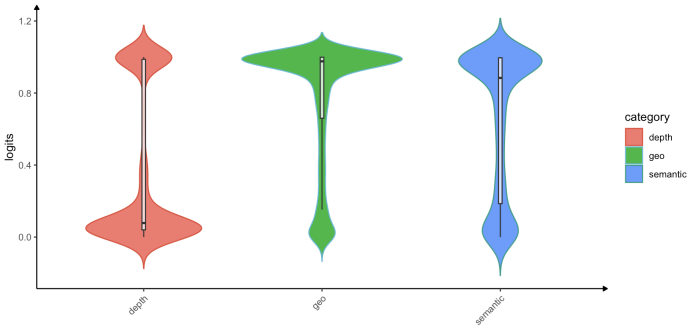


Figure 5: Voilin plot.

Discrepancy analysis

Given that normalized images are used to be input for various recent deep learning-based approaches to computer vision tasks, I will now analyze their differences in certain differents computer vision tasks via scatter plot. In my experiments, I analyze the discrepancy of the model in two computer vision tasks: image segmentation and image depth estimation. The former is a pixel-level semantic classification task, while the latter is a pixel-level depth-based classification task. Following previous research, I extracted the logical values of the true label indices corresponding to the model’s outputs for normalized input images across different tasks and recorded the corresponding predicted logical values. Then, I apply a log2 fold change operation between the model’ predicted logical values and the corresponding true label logical values. The results of the calculation are then labeled again based on their values: 0 indicates that the predicted value equals the true value and is labeled via ’Not significant’, a value greater than 1 indicates that the predicted value is greater than the value at the true label index position and is labeled via ’Up’, and a value less than 1 indicates that the predicted value is less than the value at the true label’s index position and is labeled via ’Down’. To further prevent the model comparison from being too ambiguous, I shifted all points labeled “Down” two units to the left. Regarding the definition of the Pvalue, since the range of the model’s output is between 0 and 1, I simply take the inverse of the log2 to represent the p-value. Finally, fold2change and p-value were plotted via a scatter plot, shown in the fig 6.

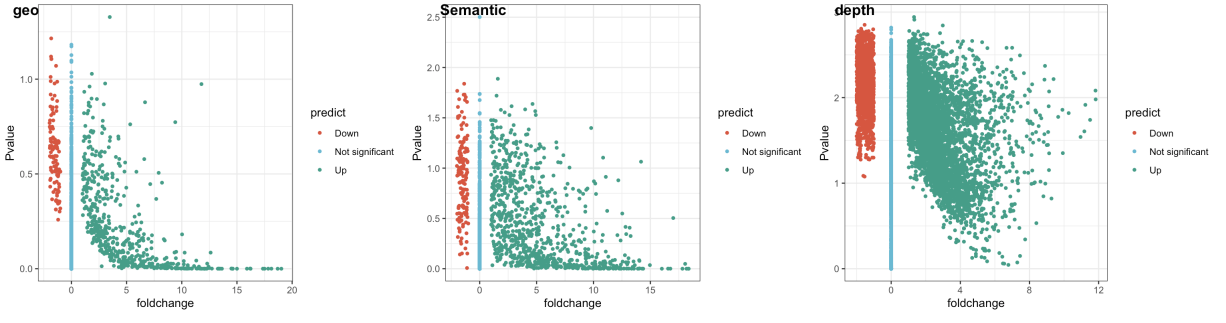


Figure 6: discrepancy plot.

Obviously, it can be seen from the scatter plot of the difference analysis that the irregular shapes of the scatter points for the same model actually follow a pattern. Moreover, this shape of regular obviously changes depending on variations in the model’s initial definition. Since the test set contained too much data, the large number of data points made it difficult to observe differences in the model’s distribution using a scatter plot. For each dataset, I randomly select a small subset for analysis. Random sampling also reveals that when a fixed random seed is used to generate random numbers, those random numbers have no effect on model training. For points labeled with ’no significant difference’—that is, those predicted correctly—I will not go into further detail in this experiment, since they were already predicted correctly. It makes little sense to further analyze whether their logical values would be closer to

the true label values, because the labels themselves were defined by themselves and are unrelated to those provided in the dataset. I defined this label so that the model can better adapt to the actual labels in its dataset. For the points with significant differences, it is obvious that the distribution of points labeled 'Down' is clearly narrower than that of points labeled 'Up'. By analyzing the points with significant differences along the horizontal axis, we can infer that, for points labeled 'Up', the closer a point is to the right, the greater the difference for that data point. For points labeled 'Down', the closer a point is also to the right, the greater the difference for that data point. Further analysis combining the training line plot and the difference scatter plot reveals that the shape of the exhibited points by outliers in the depth estimation task may be due to insufficient training.

References

[1] Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, et al. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026.

[2] Saad Wazir, Patrick Dominique Vibild, Dinh Phu Tran, Seongah Kim, and Daeyoung Kim. Medcagd: Context-aware gated decoder for efficient medical image segmentation. In *European Conference on Computer Vision*, pages 1–19. Springer, 2026.

[3] Zhongming Liu and Bingbing Jiang. Revisiting the ordering of channel and spatial attention: A comprehensive study on sequential and parallel designs, 2026.

[4] Ce Liu, Jenny Yuen, and Antonio Torralba. Sift flow: Dense correspondence across scenes and its applications. *IEEE transactions on pattern analysis and machine intelligence*, 33(5):978–994, 2010.

[5] Nathan Silberman, Derek Hoiem, Pushmeet Kohli, and Rob Fergus. Indoor segmentation and support inference from rgbd images. In *European conference on computer vision*, pages 746–760. Springer, 2012.